

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Ежов Дмитрий

Группа

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2023 г.

Задача

Мигрировать одностраничное веб-приложение, разработанное в рамках ЛР1 и ЛР2, на фреймворк Vue.JS. Приложение должно соответствовать следующим требованиям:

- Подключён клиентский роутер (Vue Router).
- Реализована работа с внешним API (предпочтительно через axios).
- Разумное разбиение интерфейса на компоненты, демонстрирующее понимание компонентного подхода.
- Использование composable-функций для выделения повторяющегося функционала в отдельные модули.

Вариант ЛР1/ЛР2 — сервис для управления проектами и командной работы (TaskerAI): аутентификация, личный кабинет (Dashboard), поиск задач с фильтрацией, страница задачи с просмотром и редактированием.

Ход работы

Стек и инструменты

В качестве основы был выбран Vue 3 с Composition API и синтаксисом `<script setup>` — это позволяет выносить логику в композаблы и переиспользовать её между компонентами. Сборка — Vite, типизация — TypeScript (`vue-tsc --noEmit` подключён к этапу `build`). Маршрутизация — Vue Router 4, HTTP-клиент — axios (переиспользован из ЛР2).

Структура проекта

src/

main.ts — точка входа, монтирование App

App.vue — корневой компонент с `<RouterView/>`

router/index.ts — 5 маршрутов с lazy-loading + beforeEach-гард

composables/ — useAuth, useTheme, useAsync, useDebounceRef

views/ — LoginView, RegisterView, DashboardView,

SearchView, TaskView

components/ — 15 SFC: AppShell, AppSidebar, AppTopbar,

SvgIcon, ThemeToggle, PasswordInput,

StatusBadge, PriorityBadge, TaskRow, TaskCard,

StatCard, ProjectItem, ActivityItem,

SubtaskItem, LabNav

api/ — слой работы с REST API (из LP2)

Маршрутизация

Настроено 5 lazy-маршрутов через динамические импорты — Vite собирает каждый view в отдельный чанк, что уменьшает начальный бандл. Маршрут `/tasks/:id` использует `props: true` — параметр передаётся в компонент как обычный prop, а не через `useRoute()`. Это отделяет компонент от роутера и упрощает тестирование.

Защита маршрутов реализована через единый `router.beforeEach`-гард, проверяющий `meta.requiresAuth` и `meta.guestOnly`. В LP2 эта логика была размазана по `utils/guard.ts` и вызывалась на каждой странице вручную — теперь она централизована.

Composable-функции

Выделено 4 композабла:

- **useAuth** — реактивный `ref<User | null>` на уровне модуля (singleton-состояние) плюс методы `login`, `logout`, `register`. Любой компонент, вызвавший `useAuth()`, видит одно и то же состояние пользователя.
- **useTheme** — управление светлой/тёмной темой через CSS-переменные и `data-theme` на `<html>`. Учитывает `prefers-color-scheme` через `matchMedia`.

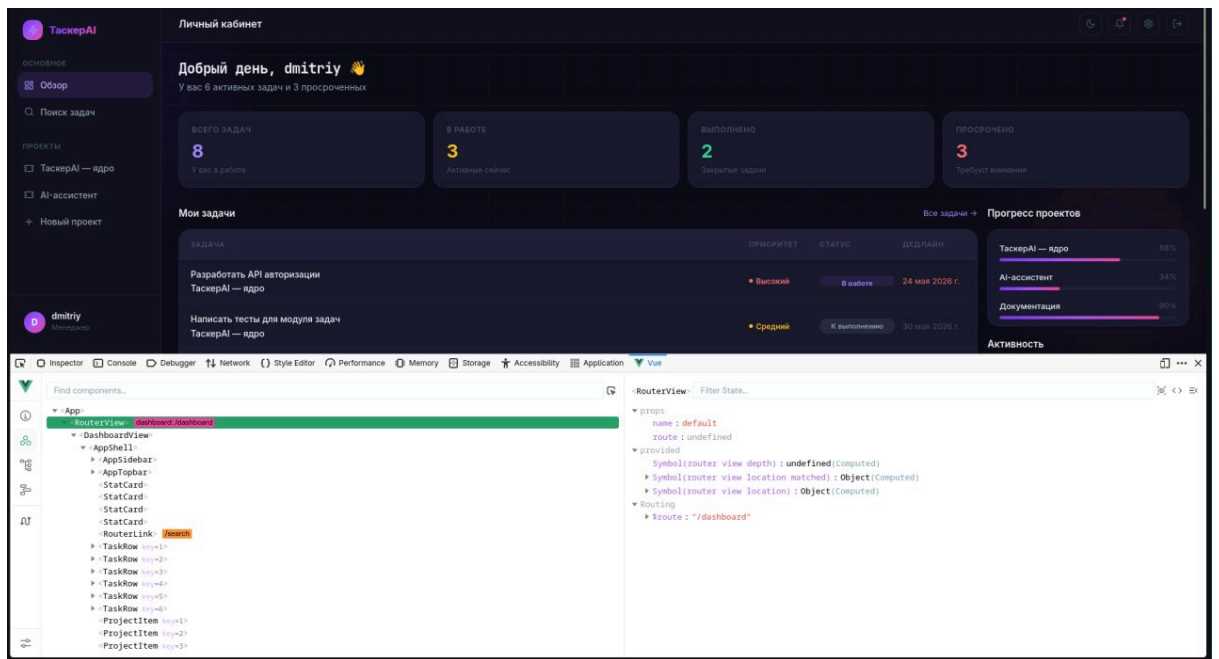
- **useAsync<T>** — обобщённый паттерн загрузки данных: возвращает `{ data, loading, error, execute }`. Внутри — **shallowRef** для данных, поскольку API-ответ заменяется целиком и глубокая реактивность не нужна.
- **useDebounceRef** — реализован через **customRef**, оборачивает сеттер в **setTimeout**. Используется в `SearchView` для дебаунса поискового запроса.

Компонентный подход

Императивная отрисовка из LP2 (например, `renderTaskRow(task)` через `document.createElement`) переведена в декларативные SFC. Каждый компонент имеет чёткий prop-API и не обращается к глобальному состоянию напрямую: данные передаются через props, события идут наверх через **emit**.

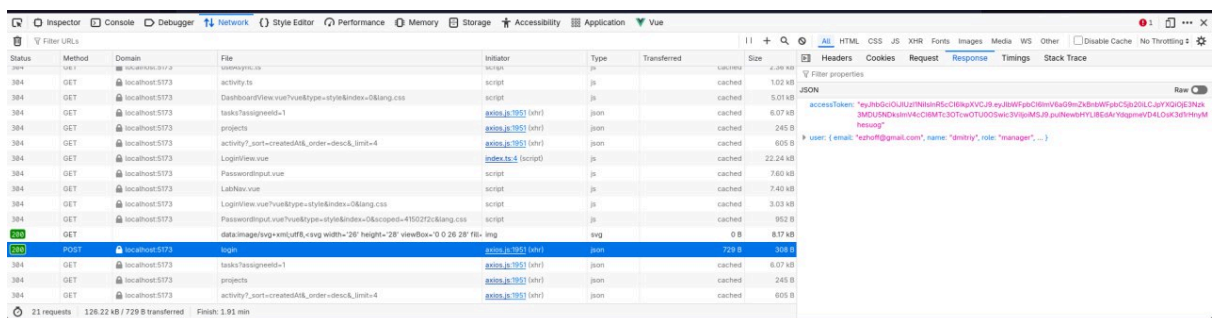
Примеры удачного разделения:

- **StatusBadge/PriorityBadge** — принимают **value** и **variant** (`'pill' | 'badge'`), не знают о бизнес-логике задач.
- **TaskRow/TaskCard** — два представления одной сущности, оба используют именованный роут `:to="{ name: 'task', params: { id } }"` — устойчиво к смене path.
- **AppShell** — layout-компонент с `<slot/>`, оборачивает **AppSidebar** и **AppTopbar**. Используется во всех views, кроме страниц аутентификации.



Работа с API

API-слой переиспользован из LP2 практически без изменений — функции возвращают **Promise** и не зависят от DOM или фреймворка. В **client.ts** (инстанс axios) добавлен interceptor на 401: при истечении токена очищается состояние **useAuth** и происходит редирект через router. Циркулярная зависимость с router разорвана через ленивый **await import('../router')** внутри интерцептора.

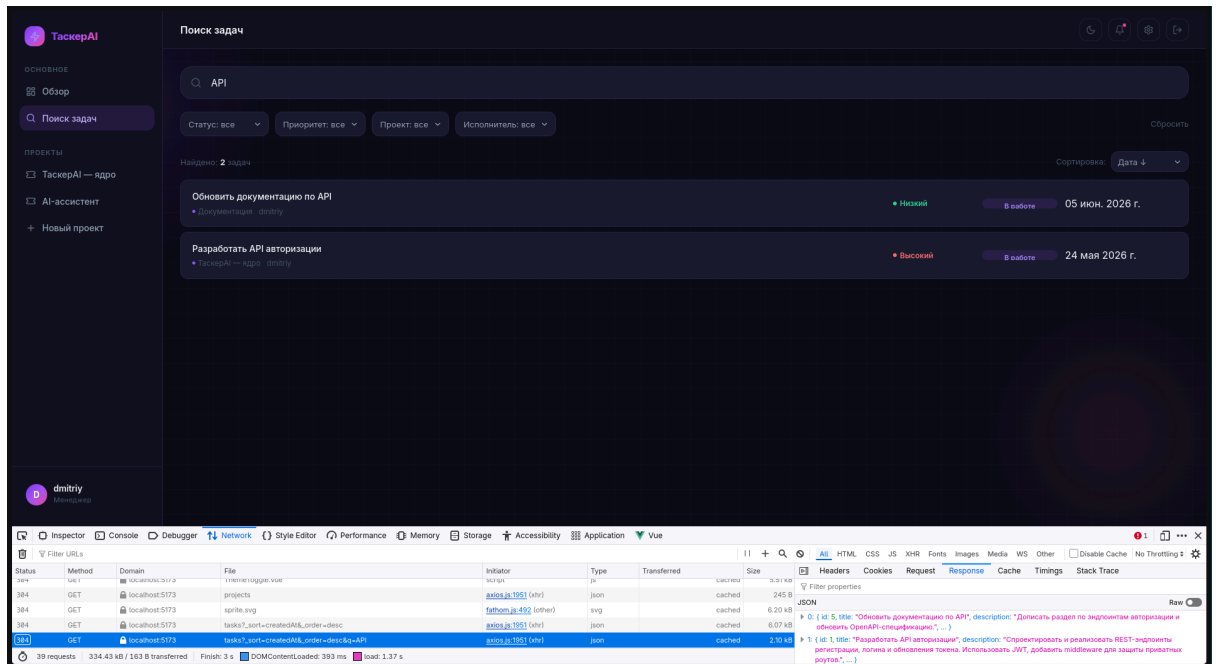


Ключевые view-компоненты

- **DashboardView** — параллельная загрузка задач и проектов через **Promise.all**, агрегаты считаются в **computed** (общее количество задач, завершённые, по статусам).
- **SearchView** — фильтры в **reactive**, два ref-а для поискового запроса: **queryInput** (мгновенно отражает ввод, привязан к

`v-model`) и `query` (дебаунсированный, триггерит `watch` для запроса к API). Это решает проблему лагающего поля при дебаунсе на самом `v-model`.

- **TaskView** — optimistic UI при редактировании: до запроса применяется патч локально, при ошибке `PATCH` состояние откатывается из backup-копии и показывается inline-сообщение через `role="alert"`.



Сравнение с ЛР2

Аспект	ЛР2 (vanilla JS)	ЛР3 (Vue)
Навигация	Полная перезагрузка между HTML-файлами	SPA с History API через RouterView
Защита маршрутов	Вызов <code>guard()</code> на каждой странице	Один <code>beforeEach</code> -гард
Обновление UI	Императивно: <code>renderTaskRow</code> , <code>appendChild</code>	Декларативно через шаблон и реактивность

Переиспользование логики	utils-модули императивными хелперами	с composables реактивным состоянием	с
--------------------------	--------------------------------------	-------------------------------------	---

Состояние пользователя	localStorage + повторное чтение на каждой странице	Singleton ref в useAuth, одна точка правды	
------------------------	--	--	--

Размер бандла	4 HTML + общий JS	1 HTML, lazy-чанки по views (main 55 KB gzip)	
---------------	-------------------	---	--

Проверка и запуск

bash

`npm install`

`npm run api` # JSON-server на 3001

`npm run dev` # Vite dev server

Вывод

В ходе работы было выполнено перепрофилирование клиентского приложения с vanilla JS на Vue 3 с Composition API. Ключевые наблюдения:

Главной сложностью оказалась не сама перепись разметки — она прямолинейна — а перевод императивного управления DOM в декларативное описание состояния. Часть данных, которые раньше жили непосредственно в DOM (видимость элементов через классы, текстовое содержимое), переехала в `ref/computed`, что потребовало переосмысления prop-API каждого компонента: он должен быть чистым, без скрытых зависимостей от глобального состояния.

Composable-функции продемонстрировали свою ценность сразу: `useAuth` как singleton-состояние избавил от необходимости вручную

синхронизировать `localStorage` между страницами; `useAsync` позволил единообразно обрабатывать `loading/error` для всех асинхронных операций; `customRef` в `useDebounceRef` показал, насколько глубоко можно интегрировать собственную логику в систему реактивности Vue.

Слоистая архитектура из ЛР2 (API-слой, отделённый от UI) подтвердила свою ценность: при смене фреймворка модули `api/*` остались практически нетронутыми. Это иллюстрирует принцип, что внешний фреймворк должен быть наименее устойчивой частью системы.

Результаты сборки: типизация чистая (`vue-tsc --noEmit` без ошибок), production-бандл составляет 143 KB JS (55 KB gzip) с разбиением на lazy-чанки по маршрутам. Все требования технического задания выполнены: подключён роутер с защищёнными маршрутами, работа с API через `axios` с `interceptor`'ами, осмысленное компонентное разбиение, четыре `composable`-функции для переиспользуемой логики.